FICHE · S11 1 PHARE CHARGE SES FICHES

Semaine 11 · Exercice 1 — Phare charge ses fiches

Module ICT 291 · Développer des interfaces utilisateurs (UI)

Durée : 30 minutes · seul · parcours essentiel et avancé

Où sont les fichiers : dans le pack, dossier `02-exercices/s11/1-phare-charge-ses-fiches/depart/`

Mandat

Les 30 lieux de Phare sont rangés dans `data.json`, comme vos fiches depuis la semaine 10. La page, elle, est vide : rien ne lui dit d'aller chercher ce fichier. Votre mission : écrire les lignes qui demandent `data.json`, lisent les fiches reçues et les montrent en cartes.

Pourquoi

C'est le même branchement que celui de votre projet, sur une page qui marche déjà. Vous le faites ici une fois, pas à pas, en regardant à chaque étape ce que le navigateur reçoit. À l'exercice 3, vous le recopiez chez vous. Le code est court ; le geste à retenir, c'est de **vérifier** chaque étape dans les outils du navigateur. La note 3 vérifie exactement cela : des cartes qui viennent de `data.json` par `fetch`, et une fiche modifiée qui change sa carte (étape 7).

Ce que vous saurez faire

- Ouvrir une page avec Live Server, et dire pourquoi un double-clic ne suffit plus.
- Demander un fichier avec `fetch`, et lire la réponse dans l'onglet Réseau.
- Transformer la réponse en fiches avec `.json()`, et compter les fiches dans la console.
- Montrer les fiches reçues avec une fonction d'affichage déjà écrite.

Les mots

- **Serveur** : un programme qui donne des fichiers quand on les lui demande. GitHub Pages en est un.
- **Live Server** : l'extension de VS Code qui fait de votre ordinateur un petit serveur. Elle ouvre votre page à une adresse qui commence par `http://127.0.0.1`, et recharge la page à chaque enregistrement.
- **`fetch`** : l'instruction JavaScript qui demande un fichier au serveur. La demande s'appelle une **requête**, ce qui revient s'appelle la **réponse**.
- **Code de statut** : le nombre que le serveur joint à sa réponse. `200` veut dire « voici le fichier », `404` veut dire « ce fichier n'existe pas ».

- **`.then`** : « quand la réponse sera arrivée, fais ceci ». La réponse met un moment à venir ; le reste de la page n'attend pas.
- **`.json()`** : la lecture du texte reçu comme du JSON. Elle rend la pile de fiches, comme en semaine 10.
- **Onglet Réseau** (Network) : l'onglet des outils du navigateur (touche F12) qui liste chaque fichier demandé par la page, avec son code de statut.
- **Console** : l'onglet des mêmes outils où s'affichent les messages de `console.log` et les erreurs. Vous l'avez ouverte en semaine 3.

Fichiers fournis

Dans le dossier `depart/` de cet exercice :

- `index.html` : la page Phare. Elle a une zone vide pour les cartes et un paragraphe vide pour les messages.
- `style.css` : les styles, déjà prêts. Vous n'y touchez pas.
- `data.json` : les 30 fiches de Phare.
- `app.js` : la fonction `afficherLieux(liste)`, qui montre une carte par fiche de la liste qu'on lui donne, et une pile `tousLesLieux` encore vide. C'est ici que vous écrivez.

Étapes

1. Installer Live Server

Dans VS Code, ouvrez le panneau **Extensions** (l'icône des quatre carrés, à gauche). Cherchez `Live Server`, choisissez celle de Ritwick Dey, puis **Installer**. Si elle est déjà installée, passez à l'étape 2.

C'est réussi si : la fiche de l'extension affiche **Désinstaller** (ou **Uninstall**) à la place d'**Installer**.

2. Ouvrir Phare

Dans le Finder ou l'Explorateur, copiez le dossier `depart/` sur votre bureau et renommez-le `s11-phare`. Glissez ce dossier sur l'icône de VS Code. **Dans VS Code**, faites un clic droit sur `index.html`, puis *Open with Live Server*.

C'est réussi si : le navigateur s'ouvre à une adresse qui commence par `http://127.0.0.1:5500`, avec le titre « Phare » et aucune carte. Aucune carte, c'est normal : rien ne demande encore `data.json`.

3. Lire ce qui existe

Dans VS Code, ouvrez `app.js`. Lisez-le sans rien changer : les deux `const` du haut, la ligne `let tousLesLieux = [];`, puis la fonction `afficherLieux`.

C'est réussi si : vous pouvez dire à votre voisin, en une phrase chacun, ce que contient `tousLesLieux` en ce moment, et ce que ferait `afficherLieux` si on lui donnait une liste de 3 fiches.

4a. Demander le fichier

Dans `app.js`, sous la ligne `// ----- Votre code de la semaine 11 commence ici -----`, ajoutez :

```
fetch("data.json")
  .then(function (reponse) {
    console.log(reponse.status);
  });
```

Enregistrez (Cmd+S ou Ctrl+S) : Live Server recharge la page tout seul. Ouvrez les outils du navigateur : touche F12 (sur Mac : Cmd+Option+I), onglet **Console**.

C'est réussi si : la console affiche `200`.

4b. Lire la ligne dans l'onglet Réseau

Dans les outils du navigateur, passez à l'onglet **Réseau** (Network) et rechargez la page (Cmd+R ou F5). Cherchez la ligne `data.json` et lisez sa colonne **État** (Status).

C'est réussi si : l'onglet Réseau montre une ligne `data.json` avec le même nombre que la console. Le fichier est arrivé ; la page ne montre encore rien, car personne ne l'a lu.

5. Lire les fiches

Dans `app.js`, remplacez votre bloc de l'étape 4a par :

```
fetch("data.json")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (donnees) {
    console.log(donnees.length);
  });
```

Le premier `.then` reçoit la réponse et la lit comme du JSON. Le second reçoit ce que la lecture a rendu : la pile de fiches, appelée ici `donnees`. Enregistrez.

C'est réussi si : la console affiche `30`. Dans l'onglet Réseau, un clic sur la ligne `data.json`, puis sur **Aperçu** (Preview), montre les fiches une à une.

6. Montrer les cartes

Dans `app.js`, dans le second `.then`, remplacez la ligne `console.log(donnees.length);` par :

```
    tousLesLieux = donnees;
    afficherLieux(tousLesLieux);
```

La première ligne range les fiches reçues dans la pile du haut du fichier. La seconde les montre. Enregistrez.

C'est réussi si : 30 cartes apparaissent. La première est « Café de la Gare ». Dans la console, tapez `tousLesLieux.length` puis Entrée : la réponse est `30`.

7. La preuve par le fichier

Dans `data.json`, changez le nom de la première fiche : `"Café de la Gare"` devient `"Café de la Gare (test)"`. Enregistrez, regardez la page. Puis remettez le nom d'origine et enregistrez.

C'est réussi si : la première carte change de nom sans que vous ayez touché à `app.js` ni à `index.html`, puis reprend son nom. Le contenu vient bien de `data.json`.

8. Le double-clic

Dans le Finder ou l'Explorateur, double-cliquez sur `index.html` du dossier `s11-phare`. Regardez l'adresse, la page, puis la console.

C'est réussi si : vous pouvez dire à votre voisin ce que montre la page, ce que dit la première ligne rouge de la console, et quelle est la différence entre les deux adresses. Fermez ensuite cet onglet et revenez à celui de Live Server.

Réussi si

- La page s'ouvre avec Live Server, à une adresse `http://127.0.0.1`.
- L'onglet Réseau montre `data.json` avec le code `200`.
- 30 cartes s'affichent, et `tousLesLieux.length` répond `30` dans la console.
- Un changement dans `data.json` se voit sur la page sans toucher au code.

Si ça bloque

Ce que vous voyez	Ce qui se passe	Ce que vous faites
Pas de *Open with Live Server* dans le clic droit	L'extension n'est pas installée, ou VS Code n'a pas ouvert le dossier entier.	Refaites l'étape 1, puis glissez le **dossier** `s11-phare` sur VS Code, pas le fichier seul.
L'adresse commence par `file://`	La page est ouverte par un double-clic.	Fermez l'onglet et rouvrez avec *Open with Live Server*.
La console ne montre rien du tout	L'enregistrement n'a pas eu lieu, ou la console a été ouverte après le chargement.	Enregistrez (un point blanc sur l'onglet du fichier veut dire « pas enregistré »), puis rechargez la page avec la console ouverte.
L'onglet Réseau est vide	Il ne note que ce qui arrive pendant qu'il est ouvert.	Laissez-le ouvert et rechargez la page.
`404` dans la console ou l'onglet Réseau	Le nom écrit dans `fetch` n'est pas celui du fichier.	Comparez lettre par lettre : `data.json`, en minuscules, à côté de `index.html`.

`afficherLieux is not defined`	Le nom de la fonction est mal écrit.	Copiez le nom depuis la ligne `function afficherLieux(liste)`.
`Assignment to constant variable`	`let tousLesLieux` est devenu `const`.	Remettez `let` : la pile change de contenu quand les fiches arrivent.
`Unexpected token` ou `missing ) after argument list`	Une parenthèse ou une accolade manque, souvent à la fin d'un `.then`.	Comparez votre bloc avec celui de l'énoncé, ligne par ligne : chaque `.then(function (…) {` se ferme par `})`.

Avec l’IA

Dans cet exercice, vous écrivez les lignes vous-même : elles sont courtes, et vous devez les avoir lues une fois pour vérifier celles de l'IA à l'exercice 3. Si une ligne reste obscure, utilisez le prompt `demander-explication-code.pdf` de votre pack, en collant seulement cette ligne et les deux qui l'entourent.

Pour les rapides

Parcours avancé : mêmes étapes, puis un ou plusieurs de ces points. Ils ne sont pas notés en plus.

- **Le résumé.** Chaque fiche a une étiquette `resume`. Ajoutez-la aux cartes, dans `afficherLieux`, avec `createElement` et `textContent`, comme les autres lignes. Vérifiez sur trois cartes.
- **L'ordre des messages.** Ajoutez `console.log("avant");` au-dessus de `fetch`, et `console.log("après");` sous tout le bloc. Rechargez et notez l'ordre des trois messages de la console (avec celui du `.then`). Expliquez-le à votre voisin.
- **La taille et le temps.** Dans l'onglet Réseau, lisez les colonnes **Taille** et **Temps** de la ligne `data.json`. Comparez avec celles de `style.css`.